

9. SET ADT

Set concepts

Set applications

A set ADT: requirements, contract

Implementations of sets: using member arrays,
linked lists, boolean arrays

Sets in the Java class library

Set concepts (1)

- A **set** is a collection of elements, with no duplicates, and in no fixed order.
 - An element does not have a “position” in the set.
- The elements of a set are usually called its **members**.
- Mathematical notation for sets:
 - $\{a, b, \dots, z\}$ is the set whose members are $a, b, \dots, z$. (The members can be written in any order.)
 - $\{ \}$ is the empty set.
 - *Note:* We can use this notation in algorithms, but it is **not** supported by Java.

Example: sets

- A set of integers:

primes = {2, 3, 5, 7, 11}

- A set of characters:

punctuation = {'.', '!', '?', ':', ';'}

- Sets of countries:

EU = {AT, BE, BG, CY, CZ, DE, DK, EE, ES, FI, FR, GR, HR, HU, IE, IT, LT, LU, LV, MT, NL, PL, PT, RO, SE, SI, SK}

NAFTA = {CA, MX, US}

NATO = {AL, BE, BG, CA, CZ, DE, DK, EE, ES, FI, FR, GR, HR, HU, IS, IT, LT, LU, LV, ME, MK, NL, NO, PL, PT, RO, SE, SI, SK, TR, UK, US}

Set concepts (2)

- The **size** (or **cardinality**) of a set s is the number of members of s . This is written $\#s$. E.g.:

$$\#NAFTA = 3$$

$$\#\{\text{red, white, red}\} = \#\{\text{red, white}\} = 2$$

duplicates are
not counted

- An **empty** set has no members.
- We can test whether x is a member of set s (i.e., s contains x). This is the **membership test**, written $x \in s$. E.g.:

$$UK \notin EU \quad) - :$$

$$AT \in EU$$

$$AT \notin NATO \quad \text{-----} \quad AT \text{ not a member of } NATO$$

Set concepts (3)

- Two sets are **equal** if they contain exactly the same members. E.g.:

$NAFTA = \{US, CA, MX\}$

order of members is not significant

$NAFTA \neq \{CA, US\}$

these sets are **not** equal

- Set s_1 **subsumes** set s_2 (or s_1 is a **superset** of s_2 , or s_2 is a **subset** of s_1) if every member of s_2 is also a member of s_1 . This is written $s_1 \supseteq s_2$. E.g.:

- $NATO \supseteq \{CA, US\}$

- $NATO \not\supseteq EU$ ----- $NATO$ does **not** subsume EU

- Equivalently $\{CA, US\} \subset NATO$ and $NATO \not\subset EU$

Set concepts (4)

- The **union** of sets s_1 and s_2 is a set containing just those values that are members of s_1 or s_2 or both. This is written $s_1 \cup s_2$. E.g.:

$$\{DK, NO, SE\} \cup \{FI, IS\} = \{DK, FI, IS, NO, SE\}$$

$$\{DK, NO, SE\} \cup \{IS, NO\} = \{DK, IS, NO, SE\}$$

- The **intersection** of sets s_1 and s_2 is a set containing just those values that are members of both s_1 and s_2 . This is written $s_1 \cap s_2$. E.g.:

$$NAFTA \cap NATO = \{CA, US\} \qquad NAFTA \cap EU = \{\}$$

- Two sets are **disjoint** if they have no common member, i.e., if their intersection is empty. E.g.:

$$NAFTA \text{ and } EU \text{ are disjoint,} \qquad NAFTA \text{ and } NATO \text{ are not disjoint}$$

- The **difference** of sets s_1 and s_2 is a set containing just those values that are members of s_1 but not of s_2 . This is written $s_1 - s_2$. E.g.:

$$NATO - EU = \{AL, CA, IS, ME, MK, NO, PL, TR, UK, US\}$$

$$EU - NATO = \{AT, CY, FI, IE, MT, SK\}$$

Set applications

- **Spell-checker:**
 - A spell-checker's dictionary is a set of words.
 - A spell-checker highlights any words in a document that are not members of its dictionary.
 - Some spell-checkers allow users to add words to their dictionaries.
- **Relational database:**
 - A relational database table is essentially a set of tuples.
 - Each tuple is distinct.
 - The tuples have no particular order.

Example: prime numbers

- A **prime number** is an integer that is divisible only by itself and 1. E.g.: 2, 3, 5, 7, 11 are prime numbers.
- **Eratosthenes' sieve algorithm:**

To compute the set of prime numbers less than n (where $n > 0$):

1. Set $sieve = \{2, 3, \dots, n-1\}$.
2. For $i = 2, 3, \dots$, while $i^2 < n$, repeat:
 - 2.1. If i is a member of $sieve$:
 - 2.1.1. Remove all multiples of i from $sieve$.
3. Terminate yielding $sieve$.

Set $sieve = \{ \}$.
 For $i = 2, \dots, n-1$, repeat:
 Add i to $sieve$.

For $m = 2 \times i, 3 \times i, \dots, n$, repeat:
 Remove m from $sieve$.

Set ADT: requirements

- Requirements:

- 1) It must be possible to make a set empty.
- 2) It must be possible to test whether a set is empty.
- 3) It must be possible to obtain the size of a set.
- 4) It must be possible to perform a membership test.
- 5) It must be possible to add or remove a member of a set.
- 6) It must be possible to test whether two sets are equal.
- 7) It must be possible to test whether one set subsumes another.
- 8) It must be possible to compute the union, intersection, or difference of two sets.
- 9) It must be possible to traverse a set.

Set ADT: contract (1)

- Possible contract for homogeneous sets:

```
public interface Set<E> {  
    // Each Set<E> object is a homogeneous set whose  
    // members are of type E.  
    ////////////////// Accessors //////////////////  
    public boolean isEmpty ();  
    // Return true if and only if this set is empty.  
    public int size ();  
    // Return the size of this set.
```

Set ADT: contract (2)

- Possible contract (*continued*):

```
public boolean contains (E it);  
// Return true if and only if it is a member of this set.  
public boolean equals (Set<E> that);  
// Return true if and only if this set is equal to that.  
public boolean containsAll (Set<E> that);  
// Return true if and only if this set subsumes that.
```

Set ADT: contract (3)

- Possible contract (*continued*):

```
////////// Transformers //////////  
public void clear ();  
// Make this set empty.  
public void add (E it);  
// Add it as a member of this set.  
// Do nothing if it is already a member of this set.  
public void remove (E it);  
// Remove it from this set.  
// Do nothing if it is not a member of this set.  
public void addAll (Set<E> that);  
// Make this set the union of itself and that.
```

Set ADT: contract (4)

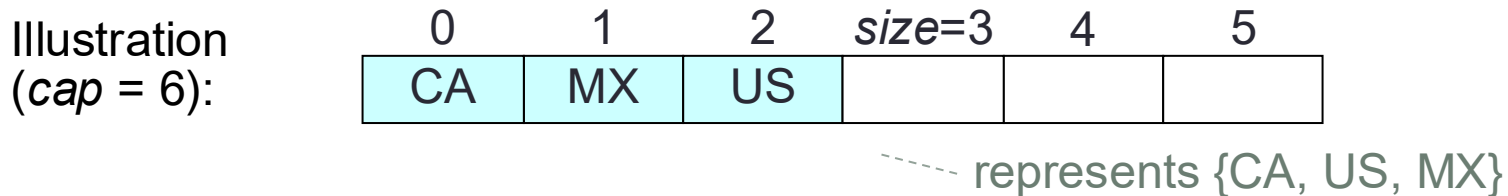
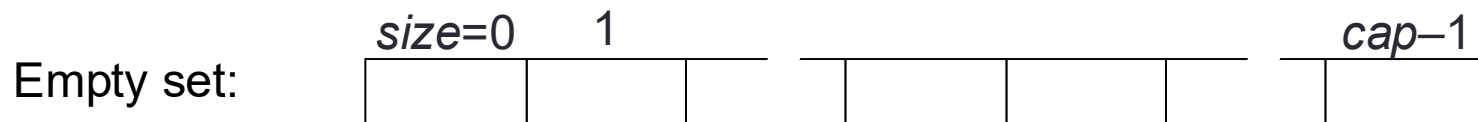
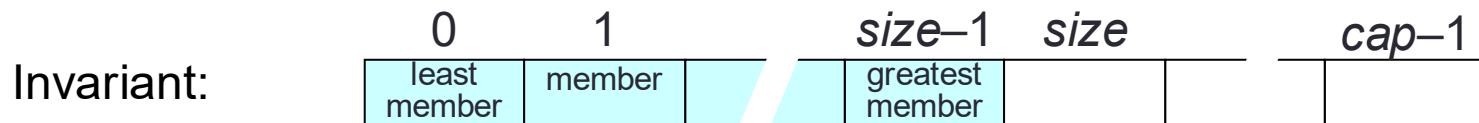
- Possible contract (*continued*):

```
public void removeAll (Set<E> that);  
// Make this set the difference of itself and that.  
public void retainAll (Set<E> that);  
// Make this set the intersection of itself and that.  
///////////////// Iterator ///////////////////  
public Iterator<E> iterator();  
// Return an iterator that will visit all members of this  
// set, in no particular order.  
}
```

NB

Implementation of sets using member arrays (1)

- Represent a *bounded* set ($\text{size} \leq \text{cap}$) by:
 - a variable *size*, containing the current size
 - A sorted array *members* of length *cap*, containing the set members in *members*[0... *size*−1], with no duplicates.



Implementation of sets using member arrays (2)

Assumptions:

- Contains, add, remove, equals, n is the size of the original set
- `containsAll`, `addAll`, `removeAll`, `retainAll`
 - last 3 operations, second set has size n'
 - Although current set is known to be implemented as an ordered array, don't know the implementation of the other
 - But we can assume both have an iterator that returns items in increasing order

See full implementation on Moodle

Implementation of sets using member arrays (3)

ContainsaAll: 是否包含另一个指定集中的所有元素

- Summary of algorithms and time complexities:

retainAll: 保留arraylist 中在指定集中也存在的那些元素，也就是删除指定集中不存在的那些元素。

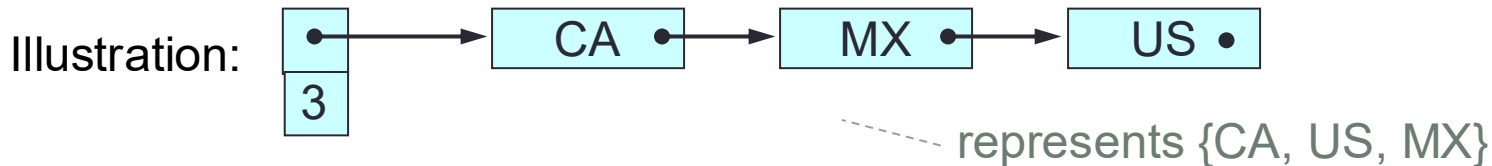
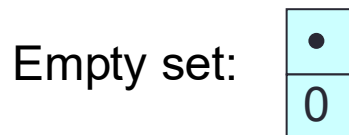
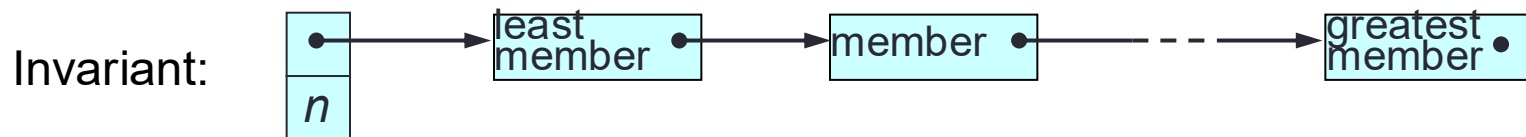
Operation	Algorithm	Time complexity
contains	binary search	$O(\log n)$
add	binary search + insertion	$O(n)$
remove	binary search + deletion	$O(n)$
equals	pairwise comparison	$O(n)$
containsAll	variant of pairwise comparison	$O(n)$
addAll	array merge	$O(n+n')$
removeAll	variant of array merge	$O(n+n')$
retainAll	variant of array merge	$O(n+n')$

where n' is the size of the other set

See full implementation on Moodle

Implementation of sets using SLLs (1)

- Represent an (unbounded) set by:
 - a variable *size*
 - A sorted SLL, containing one member per node, with no duplicates.



Implementation of sets using SLLs (2)

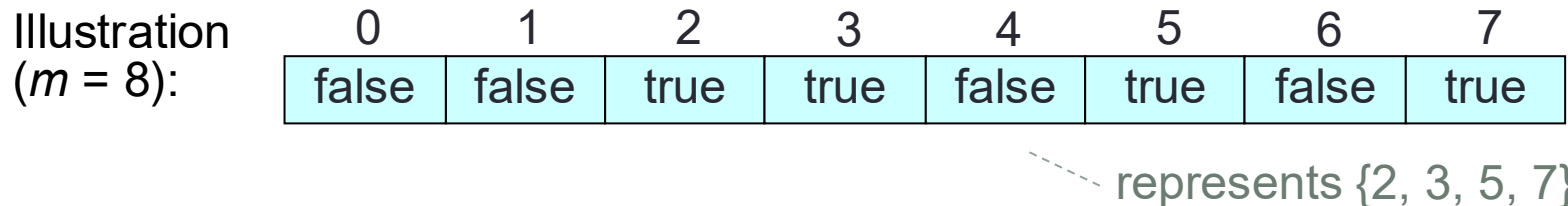
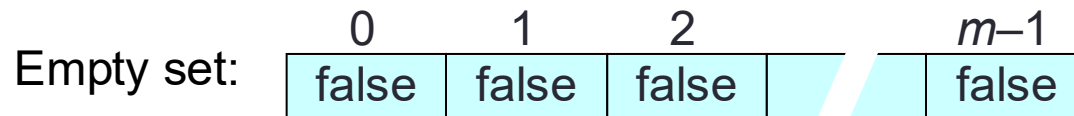
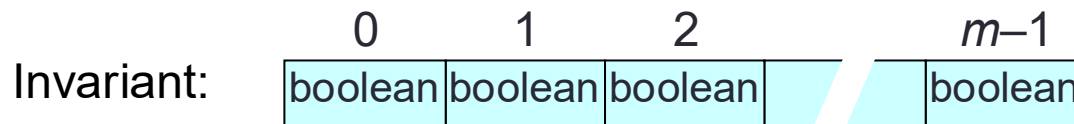
- Summary of algorithms and time complexities:

Operation	Algorithm	Time complexity
contains	SLL linear search	$O(n)$
add	SLL linear search + insertion	$O(n)$
remove	SLL linear search + deletion	$O(n)$
equals	pairwise comparison	$O(n)$
containsAll	variant of pairwise comparison	$O(n)$
addAll	SLL merge	$O(n+n')$
removeAll	variant of SLL merge	$O(n+n')$
retainAll	variant of SLL merge	$O(n+n')$

Where n is size of first set and n' is the size of the other set

Implementation of small-integer sets using boolean arrays (1)

- If the members are known to be **small integers**, in the range $0 \dots m-1$, represent the set by:
 - a boolean array b of length m , such that $b[i]$ is true if and only if i is a member of the set.



Implementation of small-integer sets using boolean arrays (2)

- Summary of algorithms and time complexities:

Operation	Algorithm	Time complexity
<code>contains</code>	test array element	$O(1)$
<code>add</code>	set array element to true	$O(1)$
<code>remove</code>	set array element to false	$O(1)$
<code>equals</code>	pairwise equality test	$O(m)$
<code>containsAll</code>	pairwise implication test	$O(m)$
<code>addAll</code>	pairwise disjunction	$O(m)$
<code>removeAll</code>	pairwise negation + conjunction	$O(m)$
<code>retainAll</code>	pairwise conjunction	$O(m)$

Comparison of set implementations (1)

Operation	Member array representation	SLL representation	Boolean array representation
contains	$O(\log n)$	$O(n)$	$O(1)$
add	$O(n)$	$O(n)$	$O(1)$
remove	$O(n)$	$O(n)$	$O(1)$
equals	$O(n)$	$O(n)$	$O(m)$
containsAll	$O(n)$	$O(n)$	$O(m)$
addAll	$O(n+n')$	$O(n+n')$	$O(m)$
removeAll	$O(n+n')$	$O(n+n')$	$O(m)$
retainAll	$O(n+n')$	$O(n+n')$	$O(m)$

Comparison of set implementations (2)

- The **member array** representation is suitable only for small or infrequently-changing sets.
- The **SLL** representation is suitable only for small sets.
- The **boolean array** representation is suitable only for sets of small integers.
- For general applications, we need a more efficient set representation: either a **search-tree** (§10) or a **hash-table** (§12).

Sets in the Java class library

- The library interface `java.util.Set<E>` is similar to the above interface `Set<E>`.
- The library class `java.util.TreeSet<E>` implements `java.util.Set<E>`, representing each set by a search-tree (§10).
- The library class `java.util.HashSet<E>` implements `java.util.Set<E>`, representing each set by a hash-table (§12).
- The library class `java.util.BitSet` represents a set of small integers by a packed boolean array. (But it does *not* implement `java.util.Set<E>`.)

Example: information retrieval (1)

- Consider a very simple information retrieval system.
- A **query** is viewed as a set of keywords.
- Given a query, the system scores each document according to how many of the keywords it contains.

Example: information retrieval (2)

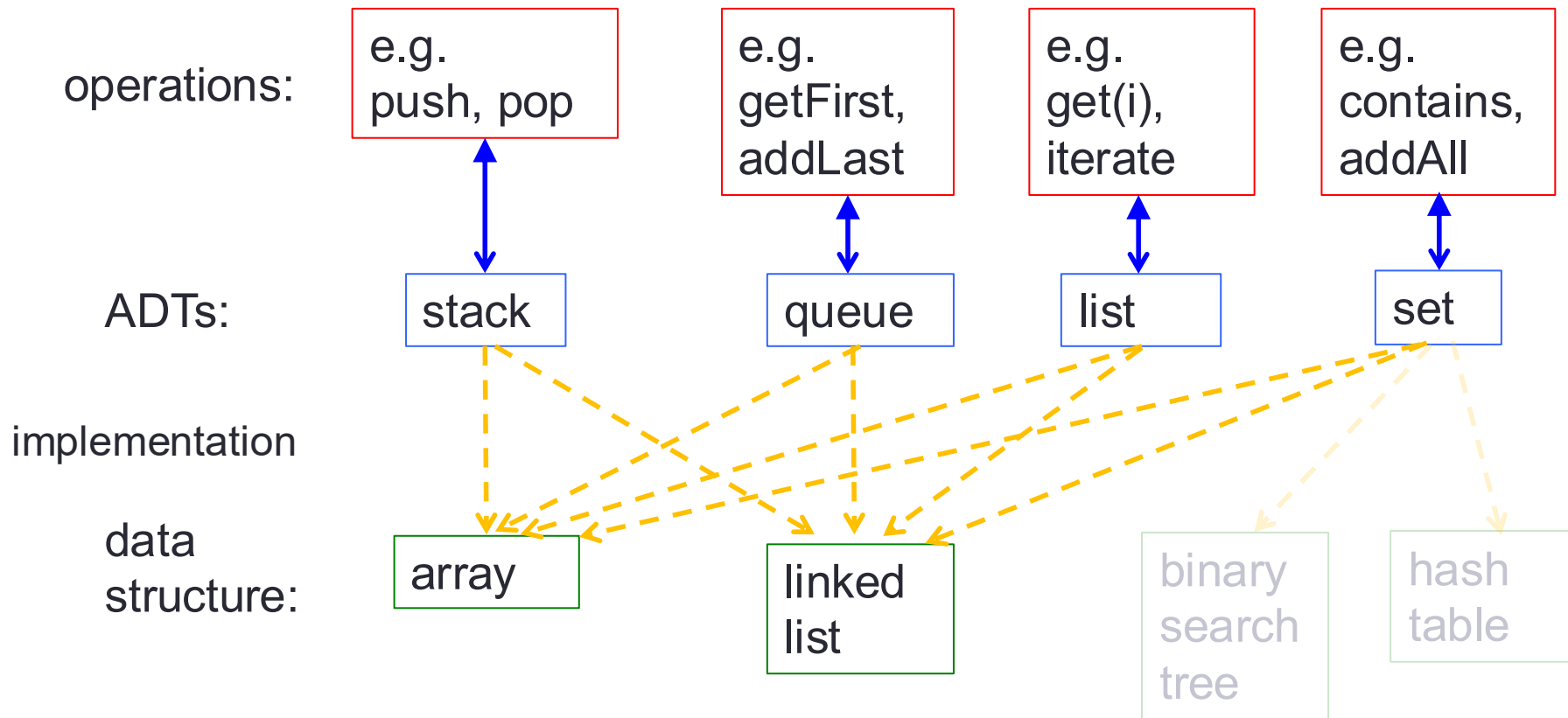
- Method for scoring a document:

```
public static int score (
    BufferedReader doc,
    Set<String> keywords) {
    // Return the number of words in keywords
    // that are also in the document doc.
    Set<String> missing = new TreeSet<String>(keywords);
    for (;;) {
        String line = doc.readLine();
        if (line == null) break;
        String[] words =
            line.split( "[^A-Z]&&[^a-z]+" );
        for (String word : words)
            missing.remove(word);
    }
    return keywords.size() - missing.size();
}
```

See complete programme

InformationRetrieval.java on
Moodle

Update on diagram



To make use of the useful features of both arrays (e.g. binary search) and linked lists (no need to bound, easy insertion) – need better data structure for set – BST (or hash table for different reasons!)